Omar Ahmed

Secret-Key Encryption

Tasks:[https://seedsecuritylabs.org/Labs_20.04/Files/Crypto_Encryption/Crypto_Encryption.pdf]

Task 1:

```
[09/28/24]seed@VM:~/.../Files$ tr 'ytnvupxzihmqjbfeargdlcskow' 'THEANDOULRISQFVPCGBYWMKXJZ' < ciphertext.txt > decoded.txt
```

```
[09/28/24]seed@VM:~/.../Files$ cat decoded.txt
THE OSCARS TURN  ON SUNDAY WHICH SEEMS ABOUT RIGHT AFTER THIS LONG STRANGE
AWARDS TRIP THE BAGGER FEELS LIKE A NONAGENARIAN TOO

THE AWARDS RACE WAS BOOKENDED BY THE DEMISE OF HARVEY WEINSTEIN AT ITS OUTSET
AND THE APPARENT IMPLOSION OF HIS FILM COMPANY AT THE END AND IT WAS SHAPED BY
THE EMERGENCE OF METOO TIMES UP BLACKGOWN POLITICS ARMCANDY ACTIVISM AND
A NATIONAL CONVERSATION AS BRIEF AND MAD AS A FEVER DREAM ABOUT WHETHER THERE
OUGHT TO BE A PRESIDENT WINFREY THE SEASON DIDNT JUST SEEM EXTRA LONG IT WAS
EXTRA LONG BECAUSE THE OSCARS WERE MOVED TO THE FIRST WEEKEND IN MARCH TO
AVOID CONFLICTING WITH THE CLOSING CEREMONY OF THE WINTER OLYMPICS THANKS
PYEONGCHANG

ONE BIG QUESTION SURROUNDING THIS YEARS ACADEMY AWARDS IS HOW OR IF THE
CEREMONY WILL ADDRESS METOO ESPECIALLY AFTER THE GOLDEN GLOBES WHICH BECAME
A JUBILANT COMINGOUT PARTY FOR TIMES UP THE MOVEMENT SPEARHEADED BY
POWERFUL HOLLYWOOD WOMEN WHO HELPED RAISE MILLIONS OF DOLLARS TO FIGHT SEXUAL
HARASSMENT AROUND THE COUNTRY

SIGNALING THEIR SUPPORT GOLDEN GLOBES ATTENDEES SWATHED THEMSELVES IN BLACK
SPORTED LAPEL PINS AND SOUNDED OFF ABOUT SEXIST POWER IMBALANCES FROM THE RED
CARPET AND THE STAGE ON THE AIR E WAS CALLED OUT ABOUT PAY INEQUITY AFTER
ITS FORMER ANCHOR CATT SADLER QUIT ONCE SHE LEARNED THAT SHE WAS MAKING FAR
LESS THAN A MALE COHOST AND DURING THE CEREMONY NATALIE PORTMAN TOOK A BLUNT
AND SATISFYING DIG AT THE ALLMALE ROSTER OF NOMINATED DIRECTORS HOW COULD
THAT BE TOPPED

AS IT TURNS OUT AT LEAST IN TERMS OF THE OSCARS IT PROBABLY WONT BE

WOMEN INVOLVED IN TIMES UP SAID THAT ALTHOUGH THE GLOBES SIGNIFIED THE
INITIATIVES LAUNCH THEY NEVER INTENDED IT TO BE JUST AN AWARDS SEASON
CAMPAIGN OR ONE THAT BECAME ASSOCIATED ONLY WITH REDCARPET ACTIONS INSTEAD
A SPOKESWOMAN SAID THE GROUP IS WORKING BEHIND CLOSED DOORS AND HAS SINCE
AMASSED  MILLION FOR ITS LEGAL DEFENSE FUND WHICH AFTER THE GLOBES WAS
FLOODED WITH THOUSANDS OF DONATIONS OF  OR LESS FROM PEOPLE IN SOME
COUNTRIES


NO CALL TO WEAR BLACK GOWNS WENT OUT IN ADVANCE OF THE OSCARS THOUGH THE
MOVEMENT WILL ALMOST CERTAINLY BE REFERENCED BEFORE AND DURING THE CEREMONY
ESPECIALLY SINCE VOCAL METOO SUPPORTERS LIKE ASHLEY JUDD LAURA DERN AND
NICOLE KIDMAN ARE SCHEDULED PRESENTERS

ANOTHER FEATURE OF THIS SEASON NO ONE REALLY KNOWS WHO IS GOING TO WIN BEST
PICTURE ARGUABLY THIS HAPPENS A LOT OF THE TIME INARGUABLY THE NAILBITER
NARRATIVE ONLY SERVES THE AWARDS HYPE MACHINE BUT OFTEN THE PEOPLE FORECASTING
THE RACE SOCALLED OSCAROLOGISTS CAN MAKE ONLY EDUCATED GUESSES

THE WAY THE ACADEMY TABULATES THE BIG WINNER DOESNT HELP IN EVERY OTHER
CATEGORY THE NOMINEE WITH THE MOST VOTES WINS BUT IN THE BEST PICTURE
CATEGORY VOTERS ARE ASKED TO LIST THEIR TOP MOVIES IN PREFERENTIAL ORDER IF A
MOVIE GETS MORE THAN  PERCENT OF THE FIRSTPLACE VOTES IT WINS WHEN NO
MOVIE MANAGES THAT THE ONE WITH THE FEWEST FIRSTPLACE VOTES IS ELIMINATED AND
ITS VOTES ARE REDISTRIBUTED TO THE MOVIES THAT GARNERED THE ELIMINATED BALLOTS
SECONDPLACE VOTES AND THIS CONTINUES UNTIL A WINNER EMERGES

IT IS ALL TERRIBLY CONFUSING BUT APPARENTLY THE CONSENSUS FAVORITE COMES OUT
AHEAD IN THE END THIS MEANS THAT ENDOFSEASON AWARDS CHATTER INVARIABLY
INVOLVES TORTURED SPECULATION ABOUT WHICH FILM WOULD MOST LIKELY BE VOTERS
SECOND OR THIRD FAVORITE AND THEN EQUALLY TORTURED CONCLUSIONS ABOUT WHICH
```

```
FILM MIGHT PREVAIL

IN  IT WAS A TOSSUP BETWEEN BOYHOOD AND THE EVENTUAL WINNER BIRDMAN
IN  WITH LOTS OF EXPERTS BETTING ON THE REVENANT OR THE BIG SHORT THE
PRIZE WENT TO SPOTLIGHT LAST YEAR NEARLY ALL THE FORECASTERS DECLARED LA
LA LAND THE PRESUMPTIVE WINNER AND FOR TWO AND A HALF MINUTES THEY WERE
CORRECT BEFORE AN ENVELOPE SNAFU WAS REVEALED AND THE RIGHTFUL WINNER
MOONLIGHT WAS CROWNED

THIS YEAR AWARDS WATCHERS ARE UNEQUALLY DIVIDED BETWEEN THREE BILLBOARDS
OUTSIDE EBBING MISSOURI THE FAVORITE AND THE SHAPE OF WATER WHICH IS
THE BAGGERS PREDICTION WITH A FEW FORECASTING A HAIL MARY WIN FOR GET OUT

BUT ALL OF THOSE FILMS HAVE HISTORICAL OSCARVOTING PATTERNS AGAINST THEM THE
SHAPE OF WATER HAS  NOMINATIONS MORE THAN ANY OTHER FILM AND WAS ALSO
NAMED THE YEARS BEST BY THE PRODUCERS AND DIRECTORS GUILDS YET IT WAS NOT
NOMINATED FOR A SCREEN ACTORS GUILD AWARD FOR BEST ENSEMBLE AND NO FILM HAS
WON BEST PICTURE WITHOUT PREVIOUSLY LANDING AT LEAST THE ACTORS NOMINATION
SINCE BRAVEHEART IN  THIS YEAR THE BEST ENSEMBLE SAG ENDED UP GOING TO
THREE BILLBOARDS WHICH IS SIGNIFICANT BECAUSE ACTORS MAKE UP THE ACADEMYS
LARGEST BRANCH THAT FILM WHILE DIVISIVE ALSO WON THE BEST DRAMA GOLDEN GLOBE
AND THE BAFTA BUT ITS FILMMAKER MARTIN MCDONAGH WAS NOT NOMINATED FOR BEST
DIRECTOR AND APART FROM ARGO MOVIES THAT LAND BEST PICTURE WITHOUT ALSO
EARNING BEST DIRECTOR NOMINATIONS ARE FEW AND FAR BETWEEN
```

Task 2:

-aes-128-cbc:

```
● [09/28/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cbc -e  -in plain.txt -out cipher.bin \-K  00112233445566778889aabbccddeeff \-iv 01
  hex string is too short, padding with zero bytes to length
● [09/28/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cbc -d  -in cipher.bin -out aescbc_decrypt.txt \-K  00112233445566778889aabbccddeef
  hex string is too short, padding with zero bytes to length
● [09/28/24]seed@VM:~/.../Labsetup$ diff plain.txt aescbc_decrypt.txt
○ [09/28/24]seed@VM:~/.../Labsetup$
```

- cbc stands for Cipher Block Chaining
- Plain text is XORed with the previous cipher block (blocks of 128 bits)
    - First block is XORed with the -iv provided
- The result of above is then encrypted using AES
- Screenshot above is saying “hex string is too short”, because it is expecting a 128 bit key (-K) and Initial Vector (-iv)

-bf-cbc:

```
● [09/28/24]seed@VM:~/.../Labsetup$ openssl enc -bf-cbc -e  -in plain.txt -out cipher.bin \-K  00112233445566778889aabbccddeeff \-iv
● [09/28/24]seed@VM:~/.../Labsetup$ openssl enc -bf-cbc -d  -in cipher.bin -out bfcbc_decrypt.txt \-K  00112233445566778889aabbccddee
● [09/28/24]seed@VM:~/.../Labsetup$ diff plain.txt bfcbc_decrypt.txt
○ [09/28/24]seed@VM:~/.../Labsetup$
```

- Stands for Blow Fish Cipher Block Chaining

- Similar to aes-128-cbc, but uses the Blow Fish algorithm to encrypt after XOR chaining.

-aes-128-cfb:

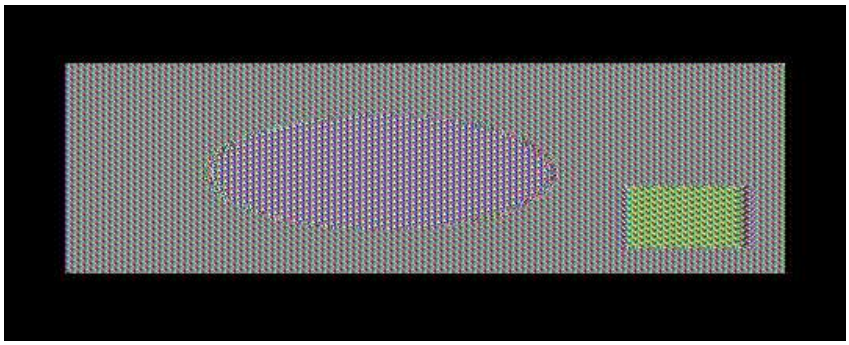
```
[09/28/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cfb -e  -in plain.txt -out cipher.bin \-K  00112233445566778889aabbccddeeff \-iv
hex string is too short, padding with zero bytes to length
[09/28/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cfb -d  -in cipher.bin -out aescfb_decrypt.txt \-K  00112233445566778889aabbccdde
hex string is too short, padding with zero bytes to length
[09/28/24]seed@VM:~/.../Labsetup$ diff plain.txt aescfb_decrypt.txt
[09/28/24]seed@VM:~/.../Labsetup$
```

- cfb stands for Cipher Feedback Mode
- The IV is first encrypted using AES, and is XORed with the first plaintext block (128 bit blocks)
- Each block is encrypted using the previous ciphertext block, so the second block will be XORed with the first blocks encryption result
    - This is repeated until the entire message is encrypted

Task 3:

EBC:

```
[09/29/24]seed@VM:~/.../Files$ openssl enc -aes-128-ec
b -e -in pic_original.bmp -out encripted_pic.bmp -K 00
112233445566778899aabbccddeeff
[09/29/24]seed@VM:~/.../Files$ head -c 54 pic_original
.bmp > header
[09/29/24]seed@VM:~/.../Files$ tail -c +55 encripted_p
ic.bmp > body
[09/29/24]seed@VM:~/.../Files$ cat header body > ecb.b
mp
```



In the encrypted image, the general structure objects can still be identified. This is because with ECB, identical blocks of plaintext are encrypted into identical blocks of ciphertext. This means that ECB is not a good idea when it comes to encrypting images.

CBC:

```
[09/29/24]seed@VM:~/.../Files$ openssl enc -aes-128-cb
c -e -in pic_original.bmp -out encripted_pic2.bmp -K 0
0112233445566778899aabbccddeeff -iv 010203040506070809
0a0b0c0d0e0f10
[09/29/24]seed@VM:~/.../Files$ tail -c +55 encripted_p
ic2.bmp > body
[09/29/24]seed@VM:~/.../Files$ cat header body > cbc.b
mp
```

Using CBC, it is impossible to tell what the original image looked like. This is because unlike in ECB, CBC makes sure that each plaintext block encrypts into different ciphertext blocks. This is done by the use of an Initialization Vector (010203040506070809 0a0b0c0d0e0f10).

When repeating the entire process using the ECB encrypted image, the result would be very similar. Using ECB will change the image however the patterns will still be visible and the encryption won't be effective. However, using CBC will be effective.

Task 4:

ECB (Electronic CodeBook): requires padding

- Because ECB encrypts in fixed-sized blocks, when the plaintext is not a multiple of the block size, padding is required

CBC (Cipher Block Chaining): requires padding

- Because CBC encrypts data in blocks of 128 bits, if the plaintext is not a multiple of this size, then padding is needed to fill in the last block

CFB (Cipher FeedBack): does not require padding

- CFB does not need padding because it does not work with fixed-sized blocks.

OFB (Output FeedBack): does not require padding

- OFB does not need padding because it does not work with fixed-sized blocks.

Size of encrypted files:

f1.txt (5 bytes) -> f1.bin (16 bytes)
f2.txt (10 bytes) -> f2.bin (16 bytes)
f3.txt (16 bytes) -> f3.bin (32 bytes)

```
[09/29/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cbc -e -in f1.txt -out f1.bin -K 00112233445566778889aabbccddeeff
hex string is too short, padding with zero bytes to length
[09/29/24]seed@VM:~/.../Labsetup$ ls -ld f1.bin
-rw-rw-r-- 1 seed seed 16 Sep 29 14:39 f1.bin
[09/29/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cbc -e -in f2.txt -out f2.bin -K 00112233445566778889aabbccddeeff
hex string is too short, padding with zero bytes to length
[09/29/24]seed@VM:~/.../Labsetup$ ls -ld f2.bin
-rw-rw-r-- 1 seed seed 16 Sep 29 14:41 f2.bin
[09/29/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cbc -e -in f3.txt -out f3.bin -K 00112233445566778889aabbccddeeff
hex string is too short, padding with zero bytes to length
[09/29/24]seed@VM:~/.../Labsetup$ ls -ld f3.bin
-rw-rw-r-- 1 seed seed 32 Sep 29 14:41 f3.bin
[09/29/24]seed@VM:~/.../Labsetup$
```

What is added to the padding during encryption:

```
[09/29/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cbc -d -in f1.bin -out new_f1.txt -K 00112233445566778889aabbccddeeff -iv
hex string is too short, padding with zero bytes to length
[09/29/24]seed@VM:~/.../Labsetup$ xxd f1.txt
00000000: 3132 3334 35                             12345
[09/29/24]seed@VM:~/.../Labsetup$ xxd new_f1.txt
00000000: 3132 3334 350b 0b0b 0b0b 0b0b 0b0b 0b0b  12345..........
[09/29/24]seed@VM:~/.../Labsetup$
```

- f1.txt padding: (pictured above)
    - 0b0b 0b0b 0b0b 0b0b 0b0b

- 0b (hex for 11, is padded). 11 bytes of data are added, because the f1.txt is 5 bytes, so 11+5 = 16.

```
[09/29/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cbc -d -in f2.bin -out new_f2.txt -K 00112233445566778889aabbccddeeff -iv
hex string is too short, padding with zero bytes to length
[09/29/24]seed@VM:~/.../Labsetup$ xxd f2.txt
00000000: 3132 3334 3536 3738 3930                1234567890
[09/29/24]seed@VM:~/.../Labsetup$ xxd new_f2.txt
00000000: 3132 3334 3536 3738 3930 0606 0606 0606  1234567890......
```

- f2.txt padding: (pictured above)
  - 0606 0606 0606
  - 6 bytes of data are added: 10+6 = 16

```
[09/29/24]seed@VM:~/.../Labsetup$ openssl enc -aes-128-cbc -d -in f3.bin -out new_f3.txt -K 00112233445566778889aabbccddeeff -iv
hex string is too short, padding with zero bytes to length
[09/29/24]seed@VM:~/.../Labsetup$ xxd f3.txt
00000000: 3132 3334 3536 3738 3930 3132 3334 3536  1234567890123456
[09/29/24]seed@VM:~/.../Labsetup$ xxd new_f3.txt
00000000: 3132 3334 3536 3738 3930 3132 3334 3536  1234567890123456
00000010: 1010 1010 1010 1010 1010 1010 1010 1010  ................
[09/29/24]seed@VM:~/.../Labsetup$
```

- f3.txt padding: (pictured above)
  - 1010 1010 1010 1010 1010 1010 1010 1010
    - 16 bytes of data are added (16+16 = 32)
- Observation is that the value of the hex byte used to pad is the number of bytes added to pad. (11 (0b) for f1, 6 (06) for f2, 16 (10) for f3)

Task 5:

Predictions:

- ECB: All data will be recovered except for the block that contains the corrupted byte. Because in ECB encryption, each block of plaintext corresponds to its respective plaintext block.
- CFB: All data will be recovered except for the block that contains the corrupted byte and the corresponding block. Because in CFB, each previous ciphertext block is used as input to encrypt the next block of plaintext. So the block with the corrupted byte will be unrecoverable. Then that block will be used to encrypt the next block, making that one unrecoverable as well.
- CBC: Sam as CFB. All data will be recoverable except for the corrupted block and the block after for the same reason.
- OFB: All data will be recovered except for the corrupted byte. Because OFB encrypts byte by byte rather than block by block. So each byte is encrypted independently from one another, leaving every uncorrupted byte unaffected.

Results:

- ECB: 16 different bytes. This means that exactly 1 block was corrupted and undecryptable.

```
 1 1234567890
 2 1234567890
 3 1234567890
 4 1234567890
 5 1234Î6£(xà+Ÿÿáÿÿ}=÷Y0
 6 1234567890
 7 1234567890
 8 1234567890
 9 1234567890
10 1234567890
11 1234567890
12 1234567890
```

- CFB: 17 different bytes. It appears that the entire corrupted block was undecryptable and only 1 byte from the corresponding block was corrupted.

```
 2 1234567890
 3 1234567890
 4 1234567890
 5 1234567890
 6 23456789µ¬Ó#£·Ö®ÿÿÿÿÝpÍéÿÿÿÿ4567890
 7 1234567890
 8 1234567890
 9 1234567890
10 1234567890
11 1234567890
```

- CBC: 17 different bytes. Same results as CFB. the entire corrupted block and only 1 byte from the corresponding block was undecryptable.

```
 2 1234567890
 3 1234567890
 4 1234567890
 5 1234t9ÿÿ¯        (dî^Jœ*ÿÿêX0
 6 1234577890
 7 1234567890
 8 1234567890
 9 1234567890
10 1234567890
11 1234567890
```

- OFB: 1 different byte. Just as anticipated, only the one corrupted byte was undecryptable and the corruption didn’t spread.

```
 1 1234567890
 2 1234567890
 3 1234567890
 4 1234567890
 5 1234567890
 6 ;234567890
 7 1234567890
 8 1234567890
 9 1234567890
10 1234567890
```

Task 6:

Task 6.1:

The ciphertext of the same file encrypted using two unique iv's are different:

```
[09/29/24]seed@VM:~/.../Labsetup$ cmp encrypted_iv1.bin encrypted_iv2.bin
encrypted_iv1.bin encrypted_iv2.bin differ: byte 1, line 1
```

The ciphertext of the same file encrypted reusing the same iv is the same each time, making it more vulnerable:

```
[09/29/24]seed@VM:~/.../Labsetup$ cmp encrypted_iv1_repeat1.bin encrypted_iv1_
peat2.bin
[09/29/24]seed@VM:~/.../Labsetup$
```

Task 6.2:

The recovered P2 is "Order: Launch a missile!"

```python
#!/usr/bin/python3

# XOR two bytearrays
def xor(first, second):
   return bytearray(x^y for x,y in zip(first, second))

MSG   = "This is a known message!"
HEX_1 = "a469b1c502c1cab966965e50425438e1bb1b5f9037a4c159"
HEX_2 = "bf73bcd3509299d566c35b5d450337e1bb175f903fafc159"

# Convert ascii string to bytearray
D1 = bytes(MSG, 'utf-8')

# Convert hex string to bytearray
D2 = bytearray.fromhex(HEX_1)
D3 = bytearray.fromhex(HEX_2)

r1 = xor(D1, D2)
r2 = xor(D2, D3)
r3 = xor(D2, D2)
print(r1.hex())
print(r2.hex())
print(r3.hex())

P2 = xor(D3, r1)
print("Recovered P2:", P2.decode('utf-8'))
~
~
~
```

```
Recovered P2: Order: Launch a missile!
[09/29/24]seed@VM:~/.../Files$ vim sample_code.py
[09/29/24]seed@VM:~/.../Files$
```

In order to get P2, given P1, C1, and C2:

- Get the keystream by xor P1 and C1
- Get P2 by xor C2 with the keystream

OFB generates a keystream independent of the plaintext and the plaintext is xored with

the keystream to create the ciphertext. This means that is the IV is reused the same keystream is generated for each encryption which allows you to recover P2 fully

In CFB, the keystream is influenced by the ciphertext as well but if the same IV is reused it can still allow you to recover parts of P2. Based on how CFB works, you would only be able to recover the first block of P2.

Task 6.3:

main.py

```python
def xor_hex_strings(hex1, hex2):
    # Convert hex strings to bytes
    b1 = bytearray.fromhex(hex1)
    b2 = bytearray.fromhex(hex2)
    # XOR the bytes
    return bytearray(x ^ y for x, y in zip(b1, b2)).hex()

# Step 1: XOR padded "Yes" with iv_bob
padded_yes_hex = "5965730d0d0d0d0d0d0d0d0d0d0d0d0d0d"
iv_bob_full = "beb230abafadb036e6b700f5d0fba3d1"

intermediate = xor_hex_strings(padded_yes_hex, iv_bob_full)

# Step 2: XOR the result with iv_next
iv_next_full = "bf001dbdafadb036e6b700f5d0fba3d1"
plaintext_to_send = xor_hex_strings(intermediate, iv_next_full)

print("Plaintext to send for 'Yes':", plaintext_to_send)
```

Ln: 10, Col: 40

Run Share Command Line Arguments

```
Plaintext to send for 'Yes': 58d75e1b0d0d0d0d0d0d0d0d0d0d0d0d0d

** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

```
[09/29/24]seed@VM:~$ nc 10.9.0.80 3000
Bob's secret message is either "Yes" or "No", without quotations.
Bob's ciphertex: 04430fa2e394726d7894a3d8605be4d6
The IV used    : beb230abafadb036e6b700f5d0fba3d1

Next IV        : bf001dbdafadb036e6b700f5d0fba3d1
Your plaintext : 58d75e1b0d0d0d0d0d0d0d0d0d0d0d
Your ciphertext: 04430fa2e394726d7894a3d8605be4d656b6493549e9022c58af412e3d6740ba

Next IV        : 928c103ab0adb036e6b700f5d0fba3d1
Your plaintext :
```

I XORed the padded plaintext yes with iv_bob, then XORed the result with iv_next. Then I submitted the resulting value to oracle, and the ciphertext matched Bob's original ciphertext, so the message was yes.

Task 7:

```
[09/29/24]seed@VM:~/.../task7$ gcc -o task7 task7.c -lcrypto
[09/29/24]seed@VM:~/.../task7$ task7 file 764aa26b55a4da654df6b19e4bce00f4ed05e09346fb0e762583cb7da2ac93a2 aabbccddeeff
the key is: Syracuse########
[09/29/24]seed@VM:~/.../task7$
```

https://drive.google.com/file/d/1W1ZOiw4g3DKttE1NM-3p9XwrRn0xTRLZ/view?usp=sharing

task7.c can be accessed with the link above ^ there are some explanations in the form of comments. The code is also screenshotted below, you may need to zoom to be able to see.

- task7.c takes the plaintext file as the first argument, the ciphertext, and the IV.
- It converts these three arguments into byte arrays
- It then opens the words.txt file provided, and checks all words that are <=16 characters long (padding those that are not 16 with #'s)
    - Uses the openssl/crypto library to encrypt the plaintext using each word as a key
    - For each key, the encrypted data is compared to the ciphertext provided, and if they match, the program prints that key and breaks.

task7 > C task7.c > main(int, char * [])

```c
#include <stdio.h>
#include <openssl/evp.h>
#include <openssl/aes.h>
#include <openssl/rand.h>
#include <string.h>

//usage:
//task7 file 764aa26b55a4da654df6b19e4bce00f4ed05e09346fb0e762583cb7da2ac93a2 aabbccddeeff00998877665544332211
void hexStringToByteArray(const char *hexString, unsigned char *byteArray, size_t byteArraySize) {
    for (size_t i = 0; i < byteArraySize; i++) {
        sscanf(hexString + 2*i, "%2hhx", &byteArray[i]);
    }
}

int main(int argc, char *argv[]){ //plaintext, ciphertext, iv

        //open plaintext file
        FILE *file, *words;
        file = fopen(argv[1], "rb"); //open in binary mode
        //convert the plaintext, ciphertext and IV into workable data structures (byte arrays)
        unsigned char plaintext[21]; //assume that the plaintext is for sure 21 chars
        fread(plaintext, 1, 21, file);
        fclose(file);

        char *ciphertext_hex = argv[2];
        char *iv_hex = argv[3];

        //byte arrays will be half the length of the original hex ones
        size_t ciphertext_length = strlen(ciphertext_hex) / 2;
        size_t iv_length = strlen(iv_hex) / 2;

        unsigned char *ciphertext = malloc(ciphertext_length);
        unsigned char *iv = malloc(iv_length);

        hexStringToByteArray(ciphertext_hex, ciphertext, ciphertext_length);
        hexStringToByteArray(iv_hex, iv, iv_length);



        words = fopen("words.txt", "r"); //open words file provided

        char key[17];
```

```c
        while (fgets(key, sizeof(key), words)) {
                // Remove newline character from the key
                key[strcspn(key, "\n")] = '\0';

                //make sure key is <=16 characters
                if (strlen(key) <= 16) {
                        //pad the string
                        memset(key + strlen(key), '#', 16 - strlen(key));
                        key[16] = '\0';

                        //new AES object
                        EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();

                        //initialize with (current) key and IV
                        EVP_EncryptInit_ex(ctx, EVP_aes_128_cbc(), NULL, (unsigned char *)key, iv);

                        //test the current key by encrypting the plaintext
                        unsigned char encryption[32]; //the test encryption
                        int length; //length of the test encryption
                        EVP_EncryptUpdate(ctx, encryption, &length, plaintext, 21);

                        int final_length = length;//length is only before EncryptFinal is called, so we use a new variable to keep track of the ent
                        EVP_EncryptFinal_ex(ctx, encryption + length, &length);
                        final_length = final_length + length;

                        //finally, check the current encrypted data to the ciphertext we have to see if the current key is the one that was used
                        if (final_length == 32 && memcmp(encryption, ciphertext, 32) == 0) {
                                printf("the key is: %s\n", key);
                                EVP_CIPHER_CTX_free(ctx); //avoid memory leaks and other problems
                                fclose(words);
                                free(ciphertext);
                                free(iv);
                                return 0;
                        }

                        EVP_CIPHER_CTX_free(ctx); //avoid memory leaks and other problems
                }
        }

        fclose(words);
        printf("error\n");
```

```
        fclose(words);
        printf("error\n");
        return 1;
}
```